Desarrollo de Aplicaciones 2

GRUPO 09

Entrega Parcial N.º 2 · Arquitectura de integración

Integrantes:		**Legajo:**	**Carrera:**
1	FLORENCIA AYALA	1113600	Ing. En Informática
3	FELIPE FORTE	1136381	Ing. En Informática
4	ARAGON JUAN	1166130	Ing. En Informática
5	LARGHERO FACUNDO	1139770	Ing. En Informática
Docente: Ruiz Matías Gabriel			
Fecha: 19/10/2026		2ndo cuatrimestre	

RABBIT · LOGÍSTICA DE ÚLTIMA MILLA

Entrega Parcial N.º 2

Arquitectura de integración y primer proceso de negocio asincrónico

> *Definir la arquitectura de integración del sistema y poner en marcha el primer proceso de negocio asincrónico.*

Lo que pide la consigna	Dónde está en este documento
Diagrama de arquitectura de integración (SOA): qué componentes internos y qué sistemas externos se conectan, y por qué canal.	Sección 1, con la figura 1 y la tabla de componentes y servicios.
Clasificación justificada de cada integración externa en síncrona (SOAP o REST) o asincrónica (cola o tópico).	Sección 2: el criterio, la tabla de las 9 integraciones y una ficha justificada por cada una.
Al menos un proceso de negocio de punta a punta con mensajería asincrónica real y broker configurado.	Sección 3: el alta de pedidos del ERP (cola punto a punto) y el aviso de cambios de estado (tópico pub/sub) sobre ActiveMQ Artemis, con la figura 2 y su verificación.

Fecha de entrega: 19/10/2026 · **Unidades:** IV y V

Sistema: Rabbit, Jakarta EE 10 sobre WildFly 41 (perfil `standalone-full`), con ActiveMQ Artemis embebido.

Repositorio: la documentación completa está en `docs/`. Este resumen sale de `docs/entregas/ENTREGA-PARCIAL-2.md`, `docs/integraciones/MensajeriaFinal.md`, `docs/integraciones/MENSAJERIA-ASINCRONICA.md` y `docs/arquitectura/DECISIONES.md` (ADR).

Qué es Rabbit

Rabbit es una plataforma de logística de última milla para comercios. Cada comercio le manda sus pedidos desde su sistema de gestión (ERP); Rabbit los recibe, les reserva el stock que el comercio tiene en sus depósitos, cobra los prepagos en el banco, los asigna a un repartidor propio o los deriva a una empresa de transporte, y avisa cada novedad al comercio y a quien recibe el pedido.

Es un único sistema (un WAR desplegado en WildFly) partido en **componentes de negocio** con interfaces explícitas: cada uno es un EJB que expone un contrato (una interfaz Java) y guarda sus propios datos. Hacia afuera, Rabbit **expone servicios** REST y **consume servicios** de otros sistemas por SOAP, REST, SMTP y HTTP. Adentro, los componentes que no necesitan esperarse se hablan a través del **broker JMS**, con una cola y un tópico.

Componentes y lo que exponen

Componente	Responsabilidad	Servicio que expone (interfaz)
Pedidos	Ciclo de vida del pedido: alta desde el ERP, confirmación, reparto, entrega, entrega fallida.	`IGestionPedidos`, `ISeguimientoPedido`; API REST para el ERP y el seguimiento público.
Comercios	Comercios clientes y sus puntos de picking.	`IConsultaComercios`
Inventario	Stock consignado en depósitos; reserva y devolución (componente *stateful*).	`IReservaStock`, `IConsultaStock`
Repartidores	Flota propia, capacidad, asignación y ubicación.	`IAsignacionRepartidores`, `IGestionRepartidores`
Ruteo	Hojas de ruta, zonas por código postal, despacho y horario estimado.	`IRuteo`, `IZonas`, `ILlegadaEstimada`
Pagos y Cobranzas	Cobros, reversas y liquidación mensual; integración con el banco.	`IRegistroCobros`
Notificaciones	Avisos al comercio y a quien recibe (portal y mail).	`INotificaciones`, `IAvisosAlDestinatario`
Transportistas	Empresas de envío externas: cotizar, derivar y seguir.	`IEnvios`, `IGestionTransportistas`
Seguridad	Cuentas, roles y quién está llamando.	`IContextoUsuario`

Sistemas externos simulados. El banco legado, el transportista legado y el transportista moderno se simulan dentro del mismo WAR, pero Rabbit los llama **por la red**, con el mismo protocolo que usaría si fueran externos: al banco y al transportista legado, por su WSDL. En una rama aparte (`desafio-heterogeneidad`) el banco está implementado en Node.js y el transportista moderno en Python, y Rabbit los consume sin cambios (ADR-030). Gmail y OpenStreetMap son servicios reales.

1. Diagrama de arquitectura de integración (SOA)

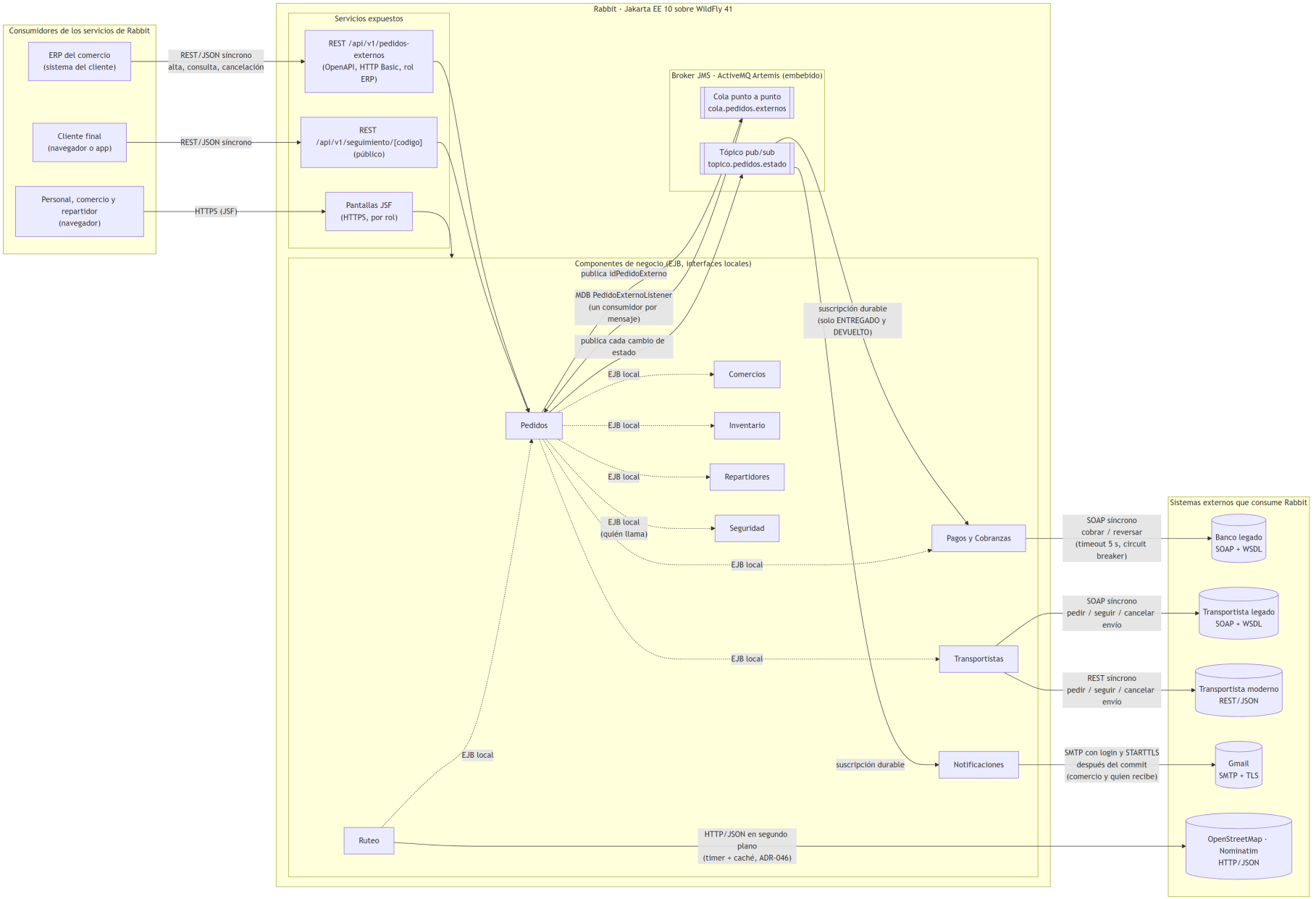


Figura 1. Consumidores de los servicios de Rabbit (izquierda), componentes internos con el broker JMS (centro) y sistemas externos que consume Rabbit (derecha), con el canal de cada conexión.

1. Cómo leer la figura 1

- **Flechas llenas entre Rabbit y el exterior:** integraciones entre sistemas, rotuladas con su canal (REST, SOAP, SMTP, HTTP).
- **Flechas hacia y desde el broker:** mensajería asincrónica real. El productor publica y sigue; no espera al consumidor.
- **Flechas punteadas:** llamadas locales entre componentes del mismo proceso, siempre por su interfaz. No son integraciones entre sistemas, pero muestran que ningún componente accede a los datos de otro por atrás.

Integraciones por lado

Lado	Sistema	Canal
Rabbit expone (otros consumen)	ERP del comercio	REST/JSON `/api/v1/pedidos-externos`, contrato OpenAPI, HTTP Basic con rol ERP
	Cliente final	REST/JSON `/api/v1/seguimiento/{codigo}`, público
	Personal, comercio y repartidor	Pantallas JSF por HTTPS, según el rol
Rabbit consume	Banco legado	SOAP + WSDL
	Transportista legado	SOAP + WSDL
	Transportista moderno	REST/JSON
	Gmail	SMTP con login y STARTTLS (puerto 587)
	OpenStreetMap (Nominatim)	HTTP/JSON, desde un timer y con caché
Adentro, por el broker	Alta del pedido externo → sincronización	Cola punto a punto `cola.pedidos.externos`
	Cambio de estado del pedido → Notificaciones y Pagos	Tópico pub/sub `topico.pedidos.estado`, suscripciones durables

2. Clasificación de cada integración

No se eligió una sola tecnología de mensajería para todo el sistema: se eligió la que corresponde a cada integración, respondiendo siempre las mismas dos preguntas.

Pregunta 1. ¿El que llama necesita la respuesta para seguir?

Sí → síncrona.

El proceso no puede avanzar sin el resultado (un cobro aprobado, un código de seguimiento).

No → asincrónica.

Alcanza con dejar el pedido de trabajo; si el otro lado está caído o lento, el mensaje espera.

Pregunta 2. ¿Cuál, dentro de cada tipo?

Síncrona:

si el otro sistema expone o exige un contrato formal (WSDL, faults tipados), SOAP; si es un partner moderno o una consulta simple, REST.

Asincrónica:

si el mensaje lo tiene que procesar uno solo, cola; si el mismo hecho le interesa a varios, tópico.

Resumen

#	Integración	Dirección	¿Necesita la respuesta?	Tipo	Canal
1	Recibir pedidos del ERP del comercio	Entrante	Sí (si se aceptó)	Síncrona	REST
2	Seguimiento público del pedido	Entrante	Sí	Síncrona	REST
3	Cobrar y reversar en el banco legado	Saliente	Sí	Síncrona	SOAP
4	Pedir, seguir y cancelar envíos (transportista legado)	Saliente	Sí	Síncrona	SOAP
5	Lo mismo con el transportista moderno	Saliente	Sí	Síncrona	REST
6	Convertir el pedido del ERP en pedido real	Interna	No	Asincrónica	Cola
7	Avisar los cambios de estado del pedido	Interna	No	Asincrónica	Tópico
8	Avisos por mail al comercio y a quien recibe	Saliente	No	Asincrónica*	SMTP (Gmail)
9	Ubicar direcciones para ordenar la hoja de ruta	Saliente	Sí, pero nadie espera	Síncrona*	REST/HTTP

* La 8 usa un protocolo de envío (SMTP) que se dispara después del commit: para el negocio es asincrónica, aunque la conexión SMTP en sí sea una conversación. La 9 es una consulta síncrona, pero corre en segundo plano y con caché: ningún usuario la espera.

2. Justificación de cada integración

1 · Recibir pedidos del ERP del comercio

SÍNCRONA · REST

¿Necesita respuesta?	Sí, pero solo una parte. El ERP necesita saber en el momento si Rabbit aceptó el pedido y con qué identificador, para poder seguirlo. No necesita esperar la reserva de stock: eso va por la cola (integración 6).
¿SOAP o REST?	Acá Rabbit expone, así que elige pensando en el consumidor. Un ERP moderno no exige WSDL ni WS-*; JSON sobre HTTP no lo obliga a generar clientes desde un contrato SOAP. SOAP solo se justificaría si el ERP exigiera WSDL o WS-Security.
Respuesta	`201 Created` con el pedido en estado *Pendiente* y la fecha prometida, o un error que dice qué campo corregir (400/422, Problem Details RFC 9457). Es 201 y no 202 porque el recurso se crea en el momento y tiene su URI.
Lo que REST deja a cargo	Idempotencia con el header `Idempotency-Key` (un reintento no duplica el pedido), contrato en OpenAPI, HTTP Basic con rol ERP sobre HTTPS y versión en la URI (`/api/v1`).

2 · Seguimiento público del pedido

SÍNCRONA · REST

¿Necesita respuesta?	Sí: quien consulta quiere ver el estado ahora. Es una lectura; no hay nada que encolar.
¿SOAP o REST?	El consumidor es un navegador o un celular: un `GET` se usa sin herramientas especiales y se puede cachear con la caché HTTP estándar. SOAP no tendría sentido para un cliente final. No expone los datos de quien recibe.

3 · Cobrar y reversar en el banco legado

SÍNCRONA · SOAP

¿Necesita respuesta?	Sí. Un pedido prepago no se puede confirmar sin saber si el banco cobró; con mensajería asincrónica quedaría confirmado "a ciegas" y habría que deshacerlo si el banco rechaza.
¿SOAP o REST?	SOAP, porque lo impone el banco: es un sistema legado que publica un WSDL (`autorizarPago` , `reversarPago`) y comunica los rechazos con un fault tipado (`PagoRechazado`). Para dinero, el contrato formal es además una ventaja.
Costo de lo síncrono	Timeout de 5 s (el pedido sigue pendiente y se puede reintentar). Circuit breaker: tras 3 fallas seguidas corta al instante y prueba de nuevo a los 30 s, para que la caída del banco no agote los hilos de Rabbit (ADR-011). Si el banco cobró pero Rabbit falla después, una transacción compensatoria reversa el cobro (ADR-010).

4 y 5 · Transportistas: legado y moderno

SÍNCRONA · SOAP y REST

¿Necesita respuesta?	Sí. Para confirmar un pedido derivado, Rabbit necesita saber en el momento si el transportista tomó el envío y con qué código de seguimiento. Si lo rechaza, el pedido no se confirma.
¿SOAP o REST?	Lo decide cada transportista: el legado publica un WSDL (`registrarEnvio` , `consultarEnvio` , `anularEnvio`), así que se le habla SOAP; el moderno, una API JSON (`POST /envios` , `GET /envios/{codigo}` …), así que REST. Es el mismo problema de negocio resuelto con dos tecnologías, cada una detrás de un Adapter con una interfaz común: Pedidos no sabe qué hay del otro lado (ADR-016).
Seguimiento	Un sistema legado no avisa, solo responde. Por eso el estado se consulta cada 15 s en segundo plano: las consultas son síncronas, pero nadie las espera (ADR-029).

6 · Convertir el pedido del ERP en pedido real

ASINCRÓNICA · COLA

¿Necesita respuesta?	No. El ERP solo necesita saber que Rabbit recibió el pedido; validar el comercio y reservar el stock puede pasar un instante después. Si lo esperara, una base lenta o un pico de pedidos frenaría al ERP.
¿Cola o tópico?	Cola: cada pedido se tiene que sincronizar **exactamente una vez**. Con un tópico, cada suscriptor recibiría el mismo pedido y el stock se reservaría dos veces. En la cola, varias instancias del consumidor compiten y cada mensaje lo procesa una sola; además escala sumando consumidores.
Lo que la cola no garantiza	El mensaje se publica recién después del commit. Si el broker falla al publicar, un timer de respaldo levanta el pedido igual. La fila se lee con bloqueo, así que la cola y el timer no la procesan dos veces.

7 · Avisar los cambios de estado del pedido

ASINCRÓNICA · TÓPICO

¿Necesita respuesta?	No. El repartidor que marca "entregado" no tiene por qué esperar a que se mande un aviso o se acredite un cobro; si Notificaciones estuviera caído, el pedido igual tiene que quedar entregado.
¿Cola o tópico?	Tópico: el mismo hecho ("el pedido 42 pasó a ENTREGADO") le interesa a **varios**, cada uno por su motivo. Hoy hay tres suscriptores: el aviso al comercio, el aviso a quien recibe y Pagos (que acredita el contra entrega o anula el cobro si el pedido vuelve al comercio). Pedidos publica sin conocerlos: sumar el aviso a quien recibe no tocó Pedidos.
Lo que el tópico no garantiza	Suscripciones **durables**: si un suscriptor está caído, el broker le guarda el mensaje. Pagos usa un **selector** y solo recibe los estados que le importan. Los mensajes repetidos o desordenados se descartan por fecha, salvo el aviso de una visita fallida, que siempre se entrega (ADR-045).

8 · Avisos por mail

ASINCRÓNICA · SMTP (GMAIL)

¿Necesita respuesta?	No. El mail sale después de confirmada la transacción: un servidor de correo caído o lento no frena ni deshace el cambio de estado, y el aviso igual queda en el portal.
Canal	SMTP, el protocolo estándar para entregar correo a cualquier casilla. Se usa una cuenta de Gmail con login y cifrado STARTTLS; las credenciales viven en la configuración del servidor, nunca en el repositorio (ADR-047).

9 · Ubicar direcciones para la hoja de ruta

SÍNCRONA · REST/HTTP

¿Necesita respuesta?	Sí, para guardar la coordenada de cada dirección. Pero la consulta corre en un timer, en segundo plano, y el resultado queda en caché: la pantalla del repartidor nunca espera a OpenStreetMap. Si una dirección todavía no se ubicó, esa parada se ordena por código postal.
Canal	Nominatim es una API pública de consulta por HTTP/JSON, sin contrato formal. Se respetan sus reglas de uso: una consulta por segundo como máximo y un User-Agent propio. Solo se envía la dirección y el código postal, nunca quién recibe (ADR-046).

3. Proceso de negocio asincrónico de punta a punta

Proceso: un comercio da de alta un pedido desde su ERP y Rabbit lo convierte en un pedido real; después, cada cambio de estado se avisa a quien le interesa. El proceso usa los **dos** estilos de mensajería: la cola punto a punto (integración 6) y el tópico pub/sub (integración 7).

Broker configurado

ActiveMQ Artemis, embebido en WildFly 41 (perfil `standalone-full.xml`, que trae el subsistema de mensajería). La propia aplicación declara la connection factory y los dos destinos con `@JMSConnectionFactoryDefinition` y `@JMSDestinationDefinition`, con conector `in-vm`: al desplegar, WildFly crea la cola y el tópico, sin configuración manual.

Destino	Tipo	Productor	Consumidores
`cola.pedidos.externos`	Cola	`PublicadorPedidosExternos`, después del commit del alta	`PedidoExternoListener` (MDB), varias instancias que compiten; cada mensaje lo procesa una sola
`topico.pedidos.estado`	Tópico	`PublicadorEstadosPedido`, en cada cambio de estado	Tres MDB con suscripción durable: aviso al comercio, aviso a quien recibe y Pagos (con selector)

Paso a paso (figura 2, en la página siguiente)

1. El ERP hace `POST /api/v1/pedidos-externos` con su `Idempotency-Key`.
2. Pedidos guarda la fila del ERP y, **después del commit**, publica `{"idPedidoExterno": N}` en la cola.
3. El ERP recibe `201 Created` con la fecha prometida: no espera la sincronización.
4. El broker le entrega el mensaje a **un** consumidor (`PedidoExternoListener`), que valida el comercio, reserva el stock y crea el pedido en estado PENDIENTE.
5. Pedidos publica "pedido N → PENDIENTE" en el tópico, y lo mismo en cada cambio posterior (confirmado, en camino, entregado, visita fallida, devuelto al comercio).
6. Cada suscriptor recibe su copia: el comercio recibe el aviso en su portal y por mail, quien recibe el pedido recibe un mail, y Pagos actúa solo cuando el pedido se entrega o vuelve al comercio.

3. Secuencia del proceso

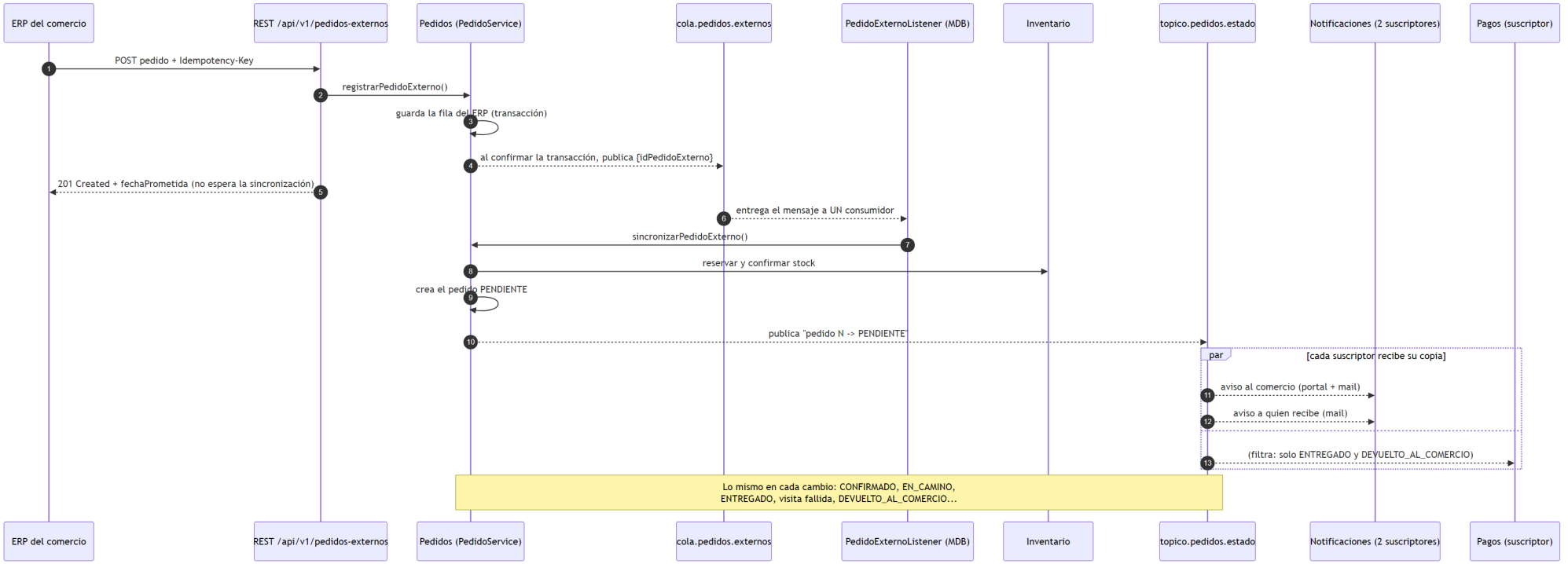


Figura 2. Alta de un pedido desde el ERP. La respuesta al ERP (paso 5 de la figura) sale antes de que el pedido se procese: la cola desacopla la sincronización y el tópico reparte cada cambio de estado a sus suscriptores.

3. Fallas contempladas y verificación

Fallas contempladas

Si pasa esto	Rabbit hace esto
El broker no está o el mensaje se pierde	Un timer (`SincronizadorDePedidos` , cada minuto) busca los pedidos del ERP sin sincronizar y los procesa. La cola es el camino rápido; el timer, la red de contención (ADR-001).
El mensaje sale pero el alta no se guardó	No puede pasar: se publica recién después del commit (ADR-002).
La cola y el timer toman el mismo pedido	La fila se lee con bloqueo y una marca de "sincronizado": el segundo la ignora.
Un suscriptor del tópico está caído	La suscripción es durable: el broker le guarda el mensaje y se lo entrega al volver (ADR-009).
Los mensajes del tópico llegan desordenados o repetidos	Notificaciones descarta lo más viejo que lo ya avisado y Pagos es idempotente. El aviso de una visita fallida se entrega aunque llegue tarde (ADR-045).
El ERP reintenta el mismo pedido	La misma `Idempotency-Key` devuelve el pedido ya creado, sin duplicarlo (ADR-018).
Falla de negocio (sin stock, comercio dado de baja)	El pedido del ERP se descarta con el motivo, que el ERP ve al consultarlo; reintentar no lo arreglaría.

Verificación (17/10/2026, WildFly 41 con base local)

- Se dieron de alta 12 pedidos por la API del ERP: todos respondieron `201` y se convirtieron en pedidos reales a través de la cola.
- Por el tópico pasaron más de 30 cambios de estado; cada uno llegó a los dos avisos (comercio y quien recibe) y, los de entrega o devolución, también a Pagos. El log muestra, en el mismo segundo, `Publicado en topico.pedidos.estado: pedido N -> …` , el aviso al comercio y el aviso a quien recibe.
- Con Gmail configurado, salieron los mails reales al comercio y a quien recibe en cada cambio, incluida una visita fallida.
- Cómo repetirlo: `docs/pruebas/FLUJO-DE-PRUEBAS.md` (secciones 4, 5, 12 y 13). La consola de WildFly (Runtime → Messaging) muestra la cola y el tópico con sus contadores.

Decisiones de arquitectura relacionadas

ADR-001 (polling de respaldo), ADR-002 (publicar después del commit), ADR-006 (tópico para los cambios de estado), ADR-009 (suscripciones durables), ADR-010 (transacción compensatoria con el banco), ADR-011 (circuit breaker), ADR-014 (consumidores competidores), ADR-016 (adapters de transportistas), ADR-018 (idempotencia), ADR-029 (seguimiento por polling), ADR-040 (avisos a quien recibe), ADR-045 (visitas fallidas), ADR-046 (OpenStreetMap), ADR-047 (mail con Gmail). Todas en `docs/arquitectura/DECISIONES.md` .